

Lab 1: ELF - Backward ldd

Author: Diana Yakupova

Email: d.yakupova@innopolis.university

Group: B23-CBS-02

Task

Implement `blld` (backward ldd) application that recursively scans directories and finds executable ELF files using specified shared libraries.

Implementation steps

Step 1: Project setup

Created modular project structure consisting of four main components:

- `cli.py` - command-line interface
- `scanner.py` - ELF parsing and dependency extraction
- `arch.py` - architecture mapping
- `report.py` - TXT/PDF report generation

Used dependencies:

- `pyelftools`
- `typer`
- `rich`
- `reportlab`

```
(.venv) root@0de69b58223a:/work# pip install -e .
Obtaining files:///work
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting pyelftools (from blld==0.1.0)
Using cached pyelftools-0.32-py3-none-any.whl.metadata (372 bytes)
Collecting typer (from blld==0.1.0)
Using cached typer-0.25.1-py3-none-any.whl.metadata (15 kB)
```

Step 2: Architecture module (`arch.py`)

Implemented `machine_to_arch()` function supporting both integer and string ELF machine identifiers from `pyelftools`.

Supported architectures:

- `x86` (`EM_386`: 3)
 - `x86_64` (`EM_X86_64`: 62)
 - `armv7` (`EM_ARM`: 40)
 - `aarch64` (`EM_AARCH64`: 183)
-

Step 3: Scanner module (`scanner.py`)

Implemented core ELF scanning logic.

`is_executable()`

Detects executable ELF files:

- `ET_EXEC` for standard executables
- `ET_DYN` with `PT_INTERP` for PIE executables
- skips shared libraries without interpreter section

`get_dependencies()`

Extracts `DT_NEEDED` entries from the `.dynamic` section.

`scan_directory()`

Recursively walks through directories, parses ELF files, extracts dependencies, and groups results by architecture.

Non-ELF and unreadable files are skipped safely without interrupting execution.

Step 4: Report module (`report.py`)

Implemented report generation in both TXT and PDF formats.

TXT report example

```
Report on dynamic used libraries by ELF executables on /bin
```

```
----- aarch64 -----  
libc.so.6 (45 execs)  
-> /bin/bash  
-> /bin/cat  
-> /bin/ls
```

PDF reports

PDF reports are generated using `reportlab` with formatted sections and grouped output.

Generated reports are stored in:

```
reports/generated/
```

Step 5: CLI module (`cli.py`)

Implemented CLI interface using `typer`.

Available options:

- `--scan-dir` - target directory
 - `-l`, `--libs` - shared library names
 - `-o`, `--output` - output report path
 - `-f`, `--format` - report format (`txt` or `pdf`)
 - `-v`, `--verbose` - debug information
 - `--help` - usage instructions
 - `--version` - application version
-

Step 6: Testing and Verification

Application was tested inside Linux Docker container on `aarch64`.

Example execution:

```
$ bldd --scan-dir /bin --libs libc.so.6 --verbose  
  
[DEBUG] printenv: arch=aarch64, deps=['libc.so.6', 'ld-linux-aarch64.so.1']  
[DEBUG] rev: arch=aarch64, deps=['libc.so.6']  
...
```

[DEBUG] Processed 654 files, 483 ELF, 482 matched

Report generated: reports/generated/bldd_report.txt

Total matched library usages: 482

Screenshots:

```
(.venv) root@de9b58223a:/work# bldd --help

Usage: bldd [OPTIONS] COMMAND [ARGS]...

Backward ldd: find ELF executables that depend on given shared libraries.

Options
  --version            Show application version.
  --scan-dir          DIRECTORY Directory to scan recursively for ELF executables. [default: /work]
  --libs              -l TEXT Shared library name(s) to search for, e.g. libc.so.6.
  --output            -o PATH Output report file path. If omitted, defaults to bldd_report.<format> in current dir.
  --format            -f TEXT Report format: txt or pdf. [default: txt]
  --verbose           -v Show progress and summary information on the console.
  --install-completion Install completion for the current shell.
  --show-completion Show completion for the current shell, to copy it or customize the installation.
  --help              Show this message and exit.

Examples: bldd --scan-dir /usr/bin --libs libc.so.6 --output report.txt bldd --scan-dir /usr/bin --libs libc.so.6 --output report.pdf --format pdf bldd --scan-dir /bin --libs libc.so.6 --verbose

(.venv) root@cc95510b5b7b:/work# bldd --scan-dir /bin --libs libc.so.6 --verbose
bldd start
Scanning /bin
Libraries: libc.so.6
Report: reports/generated/bldd_report.txt
[DEBUG] printenv: e_machine=EM_AARCH64, arch=unknown_EM_AARCH64, deps=['libc.so.6', 'ld-linux-aarch64.so.1']...
[DEBUG] rev: e_machine=EM_AARCH64, arch=unknown_EM_AARCH64, deps=['libc.so.6']...
[DEBUG] nproc: e_machine=EM_AARCH64, arch=unknown_EM_AARCH64, deps=['libc.so.6', 'ld-linux-aarch64.so.1']...
[DEBUG] dpkg-query: e_machine=EM_AARCH64, arch=unknown_EM_AARCH64, deps=['libmd.so.0', 'libc.so.6', 'ld-linux-aarch64.so.1']...
[DEBUG] sg: e_machine=EM_AARCH64, arch=unknown_EM_AARCH64, deps=['libcrypt.so.1', 'libc.so.6', 'ld-linux-aarch64.so.1']...
[DEBUG] Processed 654 files, 483 ELF, 482 matched
Report generated: reports/generated/bldd_report.txt
Total matched library usages: 482

(.venv) root@cc95510b5b7b:/work# bldd --scan-dir /bin --libs libc.so.6 --format pdf
Report generated: reports/generated/bldd_report.pdf
Total matched library usages: 482
```

Results:

- 482 executable ELF files using libc.so.6 were detected
- TXT and PDF reports were successfully generated

Step 7: Multiple libraries and Advanced usage

Tested application with multiple libraries and PDF output generation.

Examples:

```
$ bldd --scan-dir /usr/bin -l libssl.so.3 -l libcrypto.so.3 -f pdf -o ssl_report.pdf

$ bldd --scan-dir /bin -l libc.so.6 -l libcrypt.so.1 -v
```

Verified that results are sorted by executable usage count in descending order.

Requirements checklist

Requirement	Implementation	Status
Configurable directory	<code>--scan-dir option</code>	done
Configurable output	<code>--output option</code>	done
Multiple libraries	<code>-l lib1 -l lib2 ...</code>	done
Sorted by usage count	<code>sort_results()</code>	done
4+ architectures	<code>x86, x86_64, armv7, aarch64</code>	done
TXT reports	<code>generate_txt_report()</code>	done
PDF reports	<code>generate_pdf_report()</code>	done
Help command	<code>--help with examples</code>	done
Modular implementation	separate modules	done

Key technical decisions

1. Modular architecture with separate responsibilities for scanning, reporting, CLI, and architecture mapping
2. Graceful error handling for non-ELF and unreadable files
3. Usage of `pyelftools` for ELF parsing
4. Type-safe CLI implementation using `typer`
5. Architecture detection supporting both integer and string machine identifiers

Conclusion

The application successfully implements all required laboratory functionality.

The project provides:

- recursive ELF executable scanning
- shared library dependency analysis
- multi-architecture support
- TXT and PDF report generation
- configurable CLI interface

The solution was tested in Linux Docker environment on `aarch64` architecture and produced correct grouped dependency reports.